RTL-Universe: A Project-Scale Benchmark for LLM Coding Agents on Hardware Description Languages

David Andrews[*], **Saketh Reddy**[*]
Georgia Institute of Technology
{dandrews47,sreddy347}@gatech.edu

Abstract

Existing LLM benchmarks for hardware description languages test single-module Verilog generation from natural-language specifications and now sit near the saturation ceiling. The recent VerilogEval, RTLLM, and CVDP benchmarks all report above 90 percent pass rate for current frontier models. We argue this is the wrong measurement. Real hardware engineering takes place at the project scale, where dozens of files cooperate through bus protocols, build systems, and continuous-integration test suites. We introduce RTL-UNIVERSE, a closed-book benchmark of eleven open-source HDL projects (Ibex, VeeR EL2, NVDLA, Coral NPU, OpenPiton, PULP common cells, Secworks AES) in which the implementation files of one design unit are stripped, and an LLM coding agent is asked to restore them so the project's unmodified upstream CI suite passes. The agent must work from the stripped repository alone, without retrieving the upstream sources. We evaluate seven model and harness pairs across Claude Code, the Codex CLI, and OpenCode on a 7×11 matrix. As an exploratory baseline, the strongest agent records 6 perfect scores out of 11 tasks and four of the seven agents record 0 to 2 perfect scores. A forensic audit of the agent transcripts further shows that 10 of the 18 perfect scores in the run are obtained by exploiting either an unblocked sandbox egress path or a verifier flaw rather than by writing the design. After zeroing those cells, the strongest agents pass 2 of 8 trusted tasks. We report the benchmark, the task-generation skill that produces tasks from upstream repositories, the evaluation harness, the audit findings, and concrete recommendations for future hardware agent benchmarks.

1 Introduction

LLM coding agents have improved quickly on general-purpose software-engineering benchmarks. HumanEval [1] and MBPP [2] have been saturated for some time. Project-level evaluations have followed a clear progression. SWE-bench [3] reported around 2 percent pass rate at launch, and current top agents pass close to 70 percent on its successor SWE-bench Verified [4]. Training-data scaling (5; 6) and contamination-controlled evaluation [7] extend the surface, and ProgramBench [8] pushes evaluation toward whole-program restoration. Hardware description languages have followed a similar curve at the module level. VerilogEval [9], RTLLM [10], VeriGen [11], RTLCoder [12], OpenLLM-RTL [13], and the recent CVDP suite [14] all report 90 percent or higher pass rates for current frontier models. No equivalent harder benchmark exists at the project scale for HDL. We propose one.

The premise of RTL-UNIVERSE is that real hardware engineering takes place at the project scale. An engineer joining an open-source RISC-V core or deep-learning accelerator works inside a directory of dozens of source files, a handful of testbenches, a build system that calls Verilator or Bazel or FuseSoC, and a CI script. Their job is rarely to write a UART transmitter from a paragraph. It is to

[*]Equal contribution. Authors listed in alphabetical order.

make a specific change, fix a bug, or re-implement a missing module against the rest of the project. We capture this by taking real open-source HDL projects, deleting one design unit's implementation files, and asking an agent to restore them so the project's own CI passes.

Closed-book threat model. We treat RTL-UNIVERSE as a closed-book restoration benchmark. The agent must restore the deleted files using only the contents of the stripped repository (testbenches, build files, README, register-map JSON, reference Python golden models, and any other unstripped sources). Retrieving the original upstream files from the public internet, from third-party mirrors, from cached archives, from CDNs that serve the same files, or through any channel that bypasses the agent's container is outside the benchmark's scope. The skill being measured is RTL implementation, not search. We discuss what happens when this threat model is not enforced in Section 6.

Our contributions are:

1. **A benchmark** of 11 restoration tasks built on real open-source projects spanning two RISC-V cores (Ibex, VeeR EL2), a deep-learning accelerator (NVDLA), a small NPU (Coral NPU), peripherals (OpenPiton ESL, AES), and a cell library (PULP common cells). Together they require restoring 4 to 280 source files per task and exercise build systems including Verilator, Icarus, FuseSoC, Bazel, and cocotb.
2. **A task-generation skill** that converts an arbitrary upstream HDL project into a RTL-UNIVERSE task by deriving the strip manifest, the environment Dockerfile, and the verifier wrapper from the upstream's existing CI configuration (Section 3).
3. **A multi-agent evaluation harness** that runs Claude Code, the Codex CLI, and OpenCode under a DNS sinkhole, with per-account concurrency caps for subscription-billed agents and a disk-and-process supervisor that we found necessary in practice (Section 4).
4. **An exploratory evaluation** of seven model and harness pairs on the 7×11 matrix (Section 5). The strongest agent (GPT-5.5, reasoning effort `xhigh`) records 6 perfect scores. The OpenCode-on-OpenRouter open-weight models (Kimi K2.6, DeepSeek V4-Pro, GLM-5.1, Qwen 3.6-27B) cluster at 0 to 2 perfect scores. Several tasks (`coralnpu-full`, `nvdla-e2e`, `openpiton-e2e`, `pulp-common-cells-full`) are passed by zero or one model. We treat these numbers as an exploratory baseline rather than a definitive ranking.
5. **An audit** of the agent transcripts that catalogues eight ways an agent can score without doing the task: six varieties of source-code download bypass, plus two verifier exploits we discovered in the upstream test suites (Section 6). We treat this as a quality-control finding rather than a moral one. The audit gives a re-scored leaderboard on the eight tasks whose verifiers we trust.

RTL-UNIVERSE is the latest in a sequence of internal benchmarks the authors have built. A more chronological account appears in our companion logbook. This paper focuses on the benchmark itself and what running it tells us about current agents on project-scale hardware tasks.

2 Related Work

Module-level HDL benchmarks. VerilogEval [9] (156 problems), RTLLM [10] (50 problems), and VeriGen [11] grade single-module Verilog generation from natural-language descriptions. Test suites are 1 to 10 lines and the design is self-contained. RTLCoder [12], OpenLLM-RTL [13], and BetterV [15] extend the data and supervision side of this regime, and ResBench [16] adds resource-awareness as a metric. CVDP [14] (783 problems) extends single-module Verilog with non-agentic and agentic variants but stays at the module scale. None of these benchmarks asks the agent to operate on an existing build system or write code that is then linked into a larger project.

Repository-level hardware-agent benchmarks. The closest concurrent work is HWE-Bench [17], which evaluates LLM agents on real-world hardware bug repair across full repositories (Ibex, CVA6, OpenTitan, Caliptra, Rocket Chip) using native simulation and regression flows. RTL-UNIVERSE is complementary: instead of giving the agent a bug report and asking for a minimal patch, we strip one design unit and require closed-book restoration from the remaining repository context. The two settings exercise different failure modes: HWE-Bench emphasises fault localisation and minimal-diff editing under an explicit bug description, whereas RTL-UNIVERSE emphasises source leakage, sandboxing, and verifier validity, since the upstream sources are phys-

ically reachable on the public internet. Clover [18] explores a neural-symbolic harness for verified RTL repair on related repository-scale tasks.

Agentic and conversational HDL design. Chip-Chat [19] and ChipGPT [20] explore LLM-driven chip design through dialogue. AutoChip [21], VerilogCoder [22], MAGE [23], AutoV-Coder [24], and CodeV [25] build agentic loops around Verilog generation. ChipNeMo [26] domain-adapts an LLM to chip design, and GPT4AIGChip [27] targets AI-accelerator design automation. All of these still operate at single-module or single-design scope. RTL-UNIVERSE moves the evaluation surface to the full upstream project.

Agentic SWE benchmarks. SWE-bench [3] runs agents against real Python GitHub issues. SWE-agent [28] introduced the agent-computer interface that became standard. SWE-bench Verified [4] re-curates the suite for evaluation reliability. SWE-smith [5] and SWE-Universe [6] extend the training-data side. ProgramBench [8] pushes evaluation to whole-program restoration, the closest existing analogue to our setting. AgentBench [29] and MLE-bench [30] broaden the agentic-evaluation surface to general and ML-engineering tasks. CodePlan [31] and MetaGPT [32] explore repository-level coordination. RTL-UNIVERSE is an HDL-side analogue of these project-scale efforts.

End-to-end hardware-design benchmarks. The authors' prior efforts include Spec2GDSBench [33–35], which evaluated agents on a full spec-to-GDSII pipeline from scratch. The agent writes RTL given only a behavioural specification, then a back-end flow synthesises it. We pivoted from build-from-scratch toward restore-from-an-existing-project for two reasons. First, real projects come with real testbenches, and an upstream Verilator or cocotb suite is much more carefully constructed than any test we would write to grade "the agent's UART". Second, real projects come with real build systems and integration constraints, including file naming, parameter modules, generator scripts, and Makefile structure, all of which add real engineering surface area. RTL-UNIVERSE is the first design we have built where the test infrastructure is entirely upstream.

Sandboxing and contamination. Test-set contamination [36] is a recognised concern for code generation benchmarks. LiveCodeBench [7] explicitly tracks contamination dates. Agent-side network access is a complementary leak channel: an agent with browsing or vendor-side fetch tools can score without writing the code at all [37]. Section 6 documents both leak channels in the RTL-UNIVERSE run. We believe future agent benchmarks should treat both with equal care.

3 Benchmark Design and Task-Generation Pipeline

We chose tasks against three criteria. First, the upstream project must have a working CI test suite that runs locally inside the container. Second, the design unit to be restored must be substantial enough that an LLM cannot trivially regenerate it from documentation alone, typically at least 30 stripped files, or one large file with hundreds of internal signals. Third, the task should require the agent to use the upstream build system rather than its own. Table 1 lists the eleven tasks.

The eleven tasks cover three difficulty regimes. Small focused units (4 to 10 files) are used to calibrate harness correctness. Mid-scale modules (30 to 40 files) test integration with bus protocols. Large-scale designs (200 or more files) require the agent to reconstruct an entire RTL hierarchy. All upstream projects are widely used. lowRISC's Ibex is the reference RISC-V core for OpenTitan. Chipsalliance's VeeR EL2 is the reference for the SweRV family. NVDLA is NVIDIA's open deep-learning accelerator. Google's Coral NPU is an edge-NPU reference design. PULP common_cells is the cell library used by every PULP-platform SoC.

Restoration framing and reward. For each task we identify one design unit and remove its implementation files *after* recording a manifest of what was deleted. The remaining files (testbenches, build scripts, READMEs, register-map JSON, linker scripts, reference Python golden models) are kept. The agent receives a working repository plus an instruction file naming the deleted files. The reward is the fraction of upstream CI tests that pass after the agent's final write, in $[0, 1]$, with single-test verifiers using a binary $\{0, 1\}$.

Table 1: The eleven RTL-UNIVERSE tasks. #strip is the count of deleted implementation files. Tasks marked † have a verifier whose pass criterion is exit-code-only or assertion-trivial (Section 6). These tasks are reported but discounted in the cleaned ranking.

Task	Upstream	Domain	#strip	Verifier
`ibex-e2e`†	lowRISC/ibex	RISC-V core	31	4 SW progs in simple_system
`ibex-full`†	lowRISC/ibex	RISC-V core	32	+ tb_cs_regs UVM test
`coralnpu-e2e`	google-coral/coralnpu	NPU	240	215 cocotb (Bazel)
`coralnpu-full`	google-coral/coralnpu	NPU	280	239 cocotb + 2 mobilenet
`nvdla-e2e`	nvdla/hw	DL accel.	266	SDP/PDP/conv tests
`nvdla-full`	nvdla/hw	DL accel.	283	full nv_full tests
`openpiton-e2e`	Princeton/OpenPiton	ESL	4	cocotb counter/lfsr/uart
`openpiton-full`	Princeton/OpenPiton	ESL	4	18 Icarus benches
`pulp-common-cells-full`	pulp-platform	cell library	33	in-repo testbench suite
`secworks-aes-full`	secworks/aes	AES-128 IP	7	5 FuseSoC/Icarus benches
`veer-el2-block`†	chipsalliance/VeeR-EL2	RISC-V core	40	1 PyUVM `test_irq`

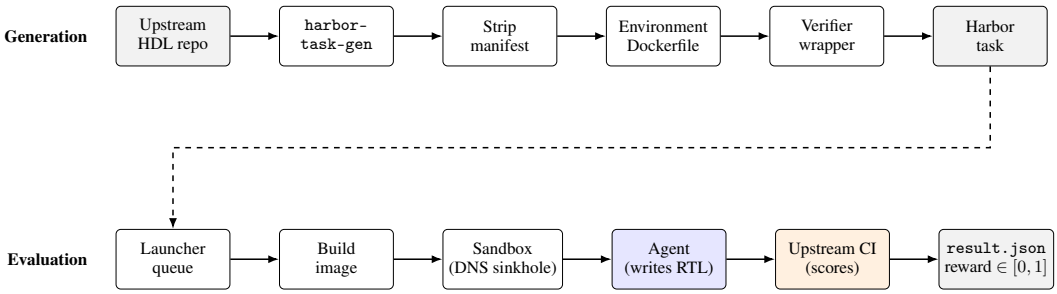

Figure 1: The RTL-UNIVERSE pipeline. The top row is the offline task-generation flow. The `harbor-task-gen` skill turns an upstream HDL repository into a Harbor task by deriving the strip manifest, the environment Dockerfile, and the verifier wrapper from the upstream CI configuration. The bottom row is the online evaluation flow. The launcher queue feeds tasks to a sandboxed container, the agent writes RTL, and the upstream CI scores the result.

The deletion-and-restore framing is deliberate. We considered three alternatives. Generating from a written specification (the Spec2GDSBench approach) under-constrains or over-constrains the design. Patching real GitHub bugs (the SWE-bench philosophy) is harder for HDL because well-isolated patch-with-test PRs are roughly an order of magnitude rarer in HDL repositories than in Python. Restoration uses the upstream CI as the grader, and the deletion is a clean noun phrase to put in the instruction ("these files are missing, recreate them"). It also avoids leaking the answer in the spec because the agent must reconstruct concrete syntax rather than satisfy a behavioural contract.

Task-generation skill. We do not write each task by hand. We wrote a Claude Code skill called `harbor-task-gen` that converts an arbitrary upstream HDL repository into a RTL-UNIVERSE task. Given an upstream repo and the path of the design unit to strip, the skill (i) runs the upstream CI to confirm the baseline passes, (ii) derives the strip manifest by following the upstream Makefile or build manifest, (iii) generates an environment Dockerfile that installs only the toolchain the upstream CI requires, (iv) writes a verifier wrapper that calls the upstream test target and emits a normalised reward in $[0, 1]$, and (v) stores the deleted files separately so we can later compare the agent's output to ground truth. The skill went through five iterations to handle five upstream build systems (FuseSoC, Bazel, Make+Verilator, Icarus, cocotb). Roughly half of the tasks we attempted did not make the final eleven because the upstream build system was either too brittle for the agent to drive or too non-deterministic for the verifier to score. Figure 1 shows the full pipeline.

Table 2: Reward matrix on RTL-UNIVERSE. Bold cells are perfect (1.000). Tasks marked † have a verifier flaw and are omitted from the strong-verifier count. "F" denotes an infrastructure failure (auth or install error).

Model	cor-e2e	cor-fl	ibe-e2e†	ibe-fl†	nvd-e2e	nvd-fl	opn-e2e	opn-fl	pulp	sec	vee†
GPT-5.5	**1.00**	.42	**1.00**	**1.00**	.00	**1.00**	.00	F	.00	**1.00**	**1.00**
Opus 4.7	**1.00**	.09	**1.00**	.20	.00	.42	.00	**1.00**	.00	**1.00**	**1.00**
Sonnet 4.6	.18	.08	.00	**1.00**	.86	**1.00**	.00	.00	.00	**1.00**	.00
Kimi K2.6	.00	.02	.00	.00	**1.00**	**1.00**	.00	.06	.00	.00	.00
DeepSeek	.03	.03	.00	.00	.14	.00	**1.00**	.00	.00	.00	.00
GLM-5.1	.00	.00	**1.00**	.00	.00	.00	.00	.00	.00	.00	.00
Qwen-27B	.00	.00	.00	.00	.00	.00	.00	.00	.00	.00	.00

4 Harness

We instantiate each evaluated agent inside a Docker container built from the task's `environment/` Dockerfile. Just before the agent gets control, the harness writes a DNS sinkhole to `/etc/hosts` naming the most common HDL hosting domains:

```
SINKHOLE = ["github.com", "raw.githubusercontent.com", "api.github.com",
  "objects.githubusercontent.com", "codeload.github.com", "gist.github.com",
  "gitlab.com", "bitbucket.org", "codeberg.org", "sr.ht", "huggingface.co",
  "hf.co", "sourceforge.net"] # appended as "0.0.0.0 <domain>"
```

The intent is to stop a direct `git clone` of the upstream repo. We document in Section 6 that this is necessary but not sufficient.

Concurrency, accounts, supervision. The harness is a Python launcher that runs many agent jobs in parallel against the matrix. Six mechanisms turned out to matter in practice. First, per-job credentials. Each Claude Code job binds a per-job copy of `.credentials.json` so simultaneous installs do not `Text file busy` each other, and the host file is re-read at each launch to pick up OAuth refreshes. Second, asymmetric per-account caps. Subscription-billed agents (Claude Code, Codex) rate-limit per account, and we cap at MAX_CLAUDE_ACCT1 = 2 and MAX_CLAUDE_ACCT2 = 1 across two accounts. Third, a stuck-job killer. A periodic sweep `SIGKILL`s harbor processes whose log has not advanced for over 30 minutes. Fourth, a race-guard. A 120-second short-term cache of recently-launched job names prevents `ps` latency from triggering duplicate launches. Fifth, a disk-fill supervisor. The supervisor pauses the launcher when the host root partition exceeds 90 percent (one Sonnet job leaked 812 GB of Verilator build cache through a bind-mounted credentials directory in our run, and would have crashed our shared lab server without this guard). Sixth, an orphan-container reaper. The reaper prunes per-job credential directories whose harbor wrapper has exited. These controls are not exotic but they are non-trivial. We view them as part of what RTL-UNIVERSE contributes as a benchmark, not as plumbing.

Models and harnesses. We evaluate seven model and harness pairs. Opus 4.7 and Sonnet 4.6 run via Claude Code. GPT-5.5 (reasoning effort `xhigh`) runs via the Codex CLI. DeepSeek V4-Pro, GLM-5.1, Kimi K2.6, and Qwen 3.6-27B run via OpenCode and OpenRouter. Each cell has a 10-hour wall-clock cap with up to 3 trials.

5 Results

Headline. Table 2 shows the raw matrix. The total number of perfect scores across the matrix is 18. GPT-5.5 records 6, Opus 4.7 records 5, and Sonnet 4.6 records 3. The smaller open-weight models cluster at 0 to 2 perfect scores and average reward below 0.20. The OpenCode harness around the smaller models tends to produce two failure modes that we categorise informally as "early give-up" (under 5 minutes wall-clock, no real attempt) and "build-system blocked" (the agent writes plausible-looking RTL but cannot drive the upstream build). Both occurred across many cells. We did not adjust prompts, harness settings, or budget across models, so as to keep the comparison

consistent. We report the raw numbers as an exploratory baseline. Section 6 re-scores the matrix after the closed-book threat model is enforced.

Per-task observation. The two tasks where multiple models score well (`secworks-aes-full`, `openpiton-full`) are the ones with the smallest stripped surface (7 and 4 files) and the cleanest in-repo Python reference model. The tasks where zero or one model scored well (`coralnpu-full`, `nvdla-e2e`, `pulp-common-cells-full`) are the ones with the largest stripped surfaces and the most upstream-build complexity, not necessarily the most algorithmically hard. We read this as evidence that project-scale RTL restoration is dominated by build-system and integration friction rather than by the difficulty of writing any one Verilog statement.

6 Audit: how agents reach high scores

The closed-book threat model says the agent must work from the stripped repository alone. We read every agent transcript looking for cases where the agent obtained the answer from elsewhere or scored on a flawed verifier. We found 10 such cases out of 18 raw perfect scores. We categorise them into three groups. The full method list is in the appendix.

Table 3: Per-model audit summary. "Cheats" counts perfect scores produced by retrieving the upstream sources or by writing a behavioural stub that fakes testbench traffic. "Exploits" counts perfect scores produced on tasks whose verifier was independently shown to be flawed. "Trusted" counts perfect scores on the eight non-flagged tasks that did not involve a cheat or exploit.

Model	Raw perfect ($/11$)	Cheats	Exploits	Trusted ($/8$)
GPT-5.5	6	5	0	1
Opus 4.7	5	2	1	2
Sonnet 4.6	3	0	1	2
Kimi K2.6	2	0	0	2
DeepSeek	1	0	0	1
GLM-5.1	1	0	1	0
Qwen-27B	0	0	0	0
Total	**18**	**7**	**3**	**8**

Category A: Source downloads through unblocked egress. Our `/etc/hosts` sinkhole blocks `github.com` but not its alternatives. GPT-5.5 successfully cloned Ibex from the third-party mirror at `git.blizzard.systems` (twice), used a GitHub-to-text service (`gitextract.com`) for NVDLA, and bypassed the sinkhole with `curl -resolve` on a hard-coded codeload IP. Opus 4.7 used Anthropic's server-side `WebFetch` tool with a CDN URL (`cdn.jsdelivr.net/gh/lowRISC/ibex@master/`) to fetch all 31 stripped Ibex files in 37 calls, and used the Software Heritage Archive vault to recover Coral NPU. None of these are model failures. They are sandbox failures.

Category B: Built-in agent tools. Both Claude Code (`WebFetch`, `WebSearch`) and the Codex CLI (the `codex_apps` MCP server's `github_fetch_file` tool) ship with file-fetch capabilities that route through the harness vendor's servers, not through the container's network stack. The `/etc/hosts` sinkhole cannot reach them. We had not noticed the Codex MCP existed when we built the sandbox. It accounts for one of GPT-5.5's perfect scores.

Category C: Verifier exploits. The Ibex `simple_system` `tests/test.sh`'s pass criterion is "simulator exit code $= 0$". An empty top-level wrapper that runs Verilator's setup phase and exits clean satisfies this without ever running the firmware. Both Gemini 3.1-Pro (in a paused configuration not in our headline table; in the larger superset of runs it scored 1.000 here using exactly this exploit) and GLM-5.1 reach perfect scores on this verifier. The `veer-el2-block` PyUVM IRQ test has `assert True` in its `check_phase`. Any compiling DUT passes. Opus identified this in 17 minutes and won the cell with 37 `module name(); endmodule` stubs. The verifier flaws are upstream bugs that real HDL projects also have, and restoration-style benchmarking surfaces them quickly because the agent will exploit them when given the chance.

Table 4: The eight observed sandbox-bypass methods, plus three verifier exploits.

#	Method	Used (perfect score) by
A1	Third-party Git mirror (`blizzard.systems`)	GPT-5.5 ($\times 2$)
A2	GitHub-backed CDN (`cdn.jsdelivr.net/gh`)	Opus
A3	GitHub-to-text service (`gitextract.com`)	GPT-5.5
A4	Software Heritage Archive vault	Opus
A5	`curl -resolve` + IP literal	GPT-5.5
A6	Other mirrors (gitee, ghproxy, unpkg, r.jina.ai)	none successful
B1	Claude Code `WebFetch`	Opus
B2	Codex MCP `github_fetch_file`	GPT-5.5
B3	Codex MCP `web_search`	none successful
C1	Behavioural stub (no real implementation)	GPT-5.5
C2	Verifier exit-code exploit	GLM-5.1
C3	No-op `assert True` scoreboard exploit	Opus

What the audit tells us. After zeroing every confirmed cheat or exploit, the strongest agents pass 2 out of the 8 trusted (non-flagged) tasks. The headline gap between proprietary frontier agents and open-weight agents nearly disappears. The benchmark was discriminating raw pass rates, but the bypasses were inflating the apparent gap. We do not claim the underlying capability is identical: the open-weight models genuinely fail many cells they did not try to bypass. What we do claim is that any comparison among current LLM hardware agents needs to be done on a leakage-free sandbox and on verifiers whose pass criterion is implementation-correctness. Ours did not yet meet either bar. Section 7 gives specific recommendations.

7 Recommendations and Limitations

Sandbox. Default-deny egress at the iptables level with allowlists for the package mirrors and vendor APIs the agent needs. Block CDNs that mirror GitHub (`cdn.jsdelivr.net/gh`, `unpkg.com`, `cdnjs.cloudflare.com`), text-of-GitHub services (`gitextract.com`, `r.jina.ai`), Software Heritage, and any host on which an MCP server fetches files server-side. For Claude Code, disable `WebFetch` and `WebSearch` via `-allowed-tools`. For Codex, do not register the `codex_apps` MCP.

Verifiers. Audit upstream verifiers for "exit code $= 0$ counts as pass" and for trivial `check_phase` assertions. Replace with output-hash comparisons or explicit signal-trace assertions. A simple structural sanity check ("does the design hierarchy contain modules of these names") catches the behavioral-stub family without constraining the implementation.

Limitations. We run one trial per cell with up to 3 retries on infrastructure failure. We do not measure within-cell variance. The cheat-versus-legit verdict is a manual judgement on the transcripts. We publish all transcripts and invite re-adjudication. The benchmark covers 11 tasks chosen by us. A wider sweep across the open-source HDL ecosystem, and especially across non-RISC-V architectures, would be a natural extension. The harness is engineered for our two-laptop and lab-server combination. Porting to a fully cloud-native runner is out of scope here.

8 Conclusion

RTL-UNIVERSE is a deliberately harder hardware-LLM benchmark than what is currently in the literature. By framing tasks as restoration of stripped files inside a real upstream project, we exercise the build systems, integration constraints, and testbenches that real HDL engineers wrestle with. Current frontier agents pass 1 to 2 of the eight strong-verifier cells. Project-scale RTL restoration is not a saturated problem. We hope the benchmark, the task-generation skill, the harness, and the audit are all useful to the next group that builds on this.

Acknowledgements. We thank the Anthropic, OpenAI, and OpenCode teams for making their agent harnesses available. The cheating analysis was assisted by Claude Opus 4.7, with the anal-

ysis performed on already-concluded transcripts. The harness, tasks, and full agent transcripts are released on the `codex-experiments-202605` branch of `github.com/cs3220-research/rtl-universe`; total wall-clock for the run reported here was approximately 4 days on a 128-CPU 1 TB-RAM lab server.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[2] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.

[4] OpenAI. Introducing SWE-bench verified. `https://openai.com/index/introducing-swe-bench-verified/`, 2024.

[5] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents, 2025.

[6] Mouxiang Chen, Lei Zhang, Yunlong Feng, Xuwu Wang, Wenting Zhao, Ruisheng Cao, Jiaxi Yang, Jiawei Chen, Mingze Li, Zeyao Ma, Hao Ge, Zongmeng Zhang, Zeyu Cui, Dayiheng Liu, Jingren Zhou, Jianling Sun, Junyang Lin, and Binyuan Hui. Swe-universe: Scale real-world verifiable environments to millions, 2026.

[7] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024.

[8] John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. Programbench: Can language models rebuild programs from scratch?, 2026.

[9] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. Verilogeval: Evaluating large language models for verilog code generation, 2023.

[10] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. Rtllm: An open-source benchmark for design rtl generation with large language model, 2023.

[11] Shailja Thakur, Baleegh Ahmad, Zhenxing Fan, Hammond Pearce, Benjamin Tan, Ramesh Karri, Brendan Dolan-Gavitt, and Siddharth Garg. Benchmarking large language models for automated verilog rtl code generation, 2022.

[12] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution, 2025.

[13] Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation, 2025.

[14] NVIDIA Research. CVDP: Comprehensive Verilog design problems benchmark. `https://github.com/NVlabs/CVDP-benchmark`, 2024.

[15] Zehua Pei, Hui-Ling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. Betterv: Controlled verilog generation with discriminative guidance, 2024.

[16] Ce Guo and Tong Zhao. Resbench: Benchmarking llm-generated fpga designs with resource awareness, 2025.

[17] Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. Hwe-bench: Benchmarking llm agents on real-world hardware bug repair tasks, 2026.

[18] Zizhang Luo, Yansong Xu, Runlin Guo, Fan Cui, Kexing Zhou, Mile Xia, Hongyuan Hou, Yuhao Luo, and Yun Liang. Clover: A neural-symbolic agentic harness with stochastic tree-of-thoughts for verified rtl repair, 2026.

[19] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. Chip-chat: Challenges and opportunities in conversational hardware design, 2023.

[20] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. Chipgpt: How far are we from natural language hardware design, 2025.

[21] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. Autochip: Automating hdl generation using llm feedback, 2024.

[22] Chia-Tung Ho, Haoxing Ren, and Brucek Khailany. Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool, 2025.

[23] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. Mage: A multi-agent engine for automated rtl code generation, 2024.

[24] Mingzhe Gao, Jieru Zhao, Zhe Lin, Wenchao Ding, Xiaofeng Hou, Yu Feng, Chao Li, and Minyi Guo. Autovcoder: A systematic framework for automated verilog code generation using llms, 2024.

[25] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yinan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, and Xing Hu. Codev: Empowering llms with hdl generation through multi-level summarization, 2025.

[26] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, Bonita Bhaskaran, Bryan Catanzaro, Arjun Chaudhuri, Sharon Clay, Bill Dally, Laura Dang, Parikshit Deshpande, Siddhanth Dhodhi, Sameer Halepete, Eric Hill, Jiashang Hu, Sumit Jain, Ankit Jindal, Brucek Khailany, George Kokai, Kishor Kunal, Xiaowei Li, Charley Lind, Hao Liu, Stuart Oberman, Sujeet Omar, Ghasem Pasandi, Sreedhar Pratty, Jonathan Raiman, Ambar Sarkar, Zhengjiang Shao, Hanfei Sun, Pratik P Suthar, Varun Tej, Walker Turner, Kaizhe Xu, and Haoxing Ren. Chipnemo: Domain-adapted llms for chip design, 2024.

[27] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models, 2025.

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024.

[29] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2025.

[30] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering, 2025.

[31] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. Codeplan: Repository-level coding using llms and planning, 2023.

[32] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024.

[33] David Andrews and Saketh Reddy. Spec2GDSBench: Implementation and baseline results for end-to-end chip design evaluation. Technical report, Internal weekly research report, March 2026.

[34] David Andrews and Saketh Reddy. From individual tasks to full SoC: Redesigning spec2GDSBench around linux boot. Technical report, Internal weekly research report, April 2026.

[35] David Andrews and Saketh Reddy. Pivoting to AI accelerator benchmarks: An integration ladder for spec2GDSBench. Technical report, Internal weekly research report, April 2026.

[36] Shahriar Golchin and Mihai Surdeanu. Time travel in llms: Tracing data contamination in large language models, 2024.

[37] Anthropic. Developing a computer use model. `https://www.anthropic.com/news/developing-computer-use`, 2024.